

Professionalism / Reflection Report

Name: Eddie Chongtham

Course: SODV2452-01, Application Development

Date: July 2026

1. What was your learning strategy in this course?

My primary strategy was to learn by building. Rather than reading documentation in isolation, I paired every concept with immediate hands-on implementation. When the course introduced Windows Forms and TCP networking, I did not stop at the provided starter code — I used it as a starting point to understand what was there, identified what was missing, and then built the missing pieces myself.

I also relied on a layered approach to understanding: I would first make something work, then step back and ask whether the code was structured well enough to be maintained or tested. This led me to apply design principles like Separation of Concerns and Single Responsibility early on, not because they were required, but because they made the code easier to reason about as the project grew.

For debugging and testing, I practised explaining code behaviour out loud before running it — predicting what a method would do, then verifying with breakpoints. This habit caught several subtle bugs before they caused problems (for example, recognising that writing a file and then the app crashing mid-write could corrupt the saved username, which led me to implement an atomic write using a temp file and rename).

2. What were the difficulties you faced and how did you overcome them?

Framework version mismatch. The starter code targeted .NET 6, which was not installed on my machine. When I tried to build, nothing worked. I overcame this by reading the error messages carefully, checking which runtimes were available with `dotnet --list-runtimes`, and updating the `.csproj` target frameworks accordingly. This taught me to never assume a specific runtime version is available and to always document the required version clearly.

Making code unit-testable without a live server. The original `ChatService` and `UserService` were difficult to test because they were tightly coupled to the network and the file system. I resolved this in two ways: for `UserService`, I added a secondary constructor that accepts a

custom storage folder, so tests could redirect all I/O to a temporary directory. For ChatService, I extracted the message formatting and validation logic into pure functions with no side effects, so tests could verify correctness without opening a TCP connection. This pattern — separating pure logic from I/O — is something I will carry into every project going forward.

Test isolation with xUnit. Initially I tried to redirect %APPDATA% via an environment variable, but this approach was fragile — it was possible for a previous test run to contaminate the environment for the next one. Switching to the injected constructor completely solved the problem: each test created its own unique temp folder and deleted it when done, with no shared state between tests.

Packaging for distribution. I had not previously packaged a .NET application for end-user installation. Learning how `dotnet publish` works — specifically the `--self-contained` and `PublishSingleFile` flags — took some trial and error. Understanding that a single-file publish bundles the entire .NET runtime into one executable was a key insight: it means the installer works on machines that have no .NET installed at all.

3. How will this course help you in your future career?

This course covered a span of the software development lifecycle that many introductory courses skip: not just writing code, but assessing existing code, gathering requirements, testing, and releasing. Each of these is a real job responsibility, not just an academic exercise.

The most transferable skills I developed are:

- **Reading and extending unfamiliar code.** The project started with starter code I did not write. Learning to read it critically — identifying what it did well and what it was missing — is exactly what happens when joining an existing team or inheriting a legacy codebase.
- **Designing for testability.** Writing code that can be unit-tested requires discipline in keeping concerns separated. Employers consistently ask candidates about testing experience; having a concrete example where I refactored a class to support injection and then wrote meaningful tests is something I can speak to directly in interviews.
- **Structured logging.** Adding Serilog and understanding how to use structured log entries for root cause analysis is a practical skill that applies to any production .NET application. Knowing how to read a log file and identify the root cause of a reported bug is a daily activity for working developers.

- **Packaging and release.** Understanding how to produce an installer that end-users can run without any technical knowledge is the difference between a project that exists on a developer's laptop and one that real people can use. This experience with Inno Setup and self-contained publishing gives me a clear picture of what a release pipeline looks like at a small scale.

Overall, this course reinforced that software development is as much about communication, structure, and process as it is about writing code — and that the habits built here (documenting decisions, writing testable code, releasing properly) are exactly what professional teams expect.